Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М71-мм

Сбор и использование данных времени выполнения для исправления ошибок с помощью встроенного в среду разработки AI-агента

Муравьев Илья Владимирович

Отчёт по производственной практике
в форме «Производственное задание»

Научный руководитель:
доцент факультета математики и компьютерных наук, к. ф.-м. н., Куликов Е. К.

Санкт-Петербург
2026

Оглавление

Введение **4**

1. Постановка задачи **7**

2. Обзор **8**

2.1. AI-агент Veai 8

2.2. Способы представления данных времени выполнения . . 10

2.2.1. Данные журналирования (логи) непроходящих тестов 11

2.2.2. Трассы выполнения 11

2.2.3. Интерактивно собираемые данные 13

2.2.4. Выбор способа представления данных времени выполнения 13

2.3. JVM-технологии сбора трасс выполнения 15

3. Архитектура **17**

3.1. Роли и связи новых и доработанных компонентов 18

3.2. Последовательность работы 20

4. Особенности реализации **23**

4.1. Сбор трассы выполнения 23

4.2. Фильтрация трассы выполнения 25

4.2.1. Присоединение асинхронных вызовов 25

4.2.2. Фокусировка на релевантных участках трассы . . 26

4.2.3. Сохранение репрезентативных вызовов 26

4.2.4. Удаление локальных повторов 27

4.2.5. Сжатие неинформативных идентификаторов объектов 27

4.2.6. Ограничение размера трассы с распределением бюджета 27

4.2.7. Суммаризация трассы выполнения с помощью большой языковой модели 28

5. Экспериментальное исследование **29**
5.1. Исследовательские вопросы 29
5.2. Метрики 29
5.3. Условия эксперимента 30
5.4. Результаты 32

Заключение **34**

Список литературы **36**

Введение

В современной индустрии разработки программного обеспечения (ПО) разработчики, тестировщики и другие участники жизненного цикла ПО всё чаще используют инструменты, основанные на технологиях искусственного интеллекта (англ. artificial intelligence, AI) [30]. Для разработчиков ПО одними из наиболее удобных в использовании инструментов являются инструменты непосредственно встраиваемые в интегрированную среду разработки (англ. integrated development environment, IDE). Сейчас на рынке сосуществует множество встраиваемых в IDE инструментов, основанных на AI, в частности предоставляемый компанией ООО «ИИТЕХ» плагин Veai [36], который встраивается в IDE, выпускаемые компанией JetBrains. Плагин Veai включает в себя тесно-интегрированного с IDE AI-агента, который позволяет пользователям в интерактивном режиме делегировать задачи разработки, тестирования и отладки искусственному интеллекту.

Одна из возникающих в процессе разработки ПО задач — это задача исправления ошибок (дефектов) реализации, найденных автоматизированными тестами (автотестами). До данной работы AI-агент Veai уже во многих случаях корректно решал эту задачу, но в ряде случаев AI-агент Veai предлагал неработающие или некорректные исправления. В частности, распространённой проблемой являлось «симптоматическое лечение» тестов, а именно: внесение изменений в код рядом с тем местом, где проявлялась ошибка реализации (где было выброшено исключение), когда настоящей причиной падения теста (англ. root cause [12]) было произошедшее существенно ранее нарушение внутренних инвариантов отлаживаемого приложения. Применение такого «симптоматического лечения» даже в небольшой доли случаев крайне опасно для долгосрочной стабильности и поддерживаемости ПО, так как «симптоматическое лечение» не устраняет нарушение инвариантов, а лишь скрывает отдельное проявление ошибки реализации.

Рассмотрим пример: в проекте `openmrs-core` на момент коммита `150ee0a` тесты завершались (падали) с ошибкой «`UnchangeableObject`

`Editing the fields [referenceRange] on Obs is not allowed»`, разработчики проекта исправили эту ошибку, предотвратив изменения уже сохранённых объектов типа `Obs` (см. рисунок 1a), в то время, как AI-агенты, такие как исходная версия AI-агента Veai, «исправляют» эту ошибку путём добавления поля `referenceRange` в список полей, которые можно редактировать (см. рисунок 1b). Что же мешает AI-агентам реализовать исправление, аналогичное исправлению от разработчиков `openmrs-core`? Одна из проблем заключается в том, что неразрешенная модификация `Obs` не упоминается в трассе стека (англ. stacktrace) исключения, отделена от непрошедшего (падающего) теста 97 кадрами стека вызовов и выполняется из класса `ObsValidator`, который содержит корректную логику инициализации новых экземпляров класса `Obs`. Естественным способом решения этой проблемы является включение в контекст AI-агента некоторых данных времени выполнения, достаточных для выявления фрагмента кода, где происходит нарушение инвариантов системы.

Данная работа посвящена сбору таких данных времени выполнения и использованию их для повышения точности исправления AI-агентом Veai ошибок, найденных автотестами. Чтобы повысить сфокусированность, работа ограничивается расширением возможностей AI-агента исключительно в среде разработки IntelliJ IDEA [15], ориентированной на JVM-языки. Была выбрана именно эта среда разработки, так как на момент начала работы для этой среды разработки был реализован наиболее полный набор функциональности Veai [37].

```
…/validator/ObsValidator.java      +2 -0
+ if (obs.getId() == null) {
      var obsRefRange = new ObsReferenceRange();

  @@ -421,3 +422,3 @@

      obs.setReferenceRange(obsRefRange);
+ }
```

(a) Исправление, выполненное разработчиками `openmrs-core`.[1]

```
…/ImmutableObsInterceptor.java      +1 -1
  private static final String[] MUTABLE_PROPS = {
      "voided",
      "dateVoided",
      "voidedBy", "voidReason",
-     "groupMembers"
+     "groupMembers", "referenceRange"
  };
```

(b) Исправление, выполненное изначальной версией AI-агента Veai при использовании модели GPT-5.2.

Рис. 1: Сравнение исправлений ошибки «`UnchangeableObject Editing the fields [referenceRange] on Obs is not allowed`» из проекта `openmrs-core`.

[1]Источник: https://github.com/openmrs/openmrs-core/compare/150ee0aed92c..d85c13706ec4.

1 Постановка задачи

Целью работы является повышение точности исправления ошибок, выявленных автотестами в проектах на JVM-языках, путём интеграции данных времени выполнения в процесс работы AI-агента Veai. Для достижения этой цели были поставлены следующие задачи.

1. Выполнить сравнительный анализ подходов и технологий сбора данных времени выполнения и выбрать подход и технологию, наиболее подходящие для использования в Veai.

2. Спроектировать архитектуру решения, позволяющую встроить в AI-агента Veai сбор и фильтрацию данных времени выполнения.

3. Реализовать и встроить в AI-агента Veai спроектированное решение.

4. Измерить, как данные времени выполнения влияют на точность исправления ошибок в реальных проектах.

2 Обзор

В данной главе проводится обзор основных возможностей AI-агента Veai и способов представления данных времени выполнения, а также выбирается технология для сбора данных времени выполнения.

2.1 AI-агент Veai

Veai — это плагин, разрабатываемый для IDE, выпускаемых компанией JetBrains и их российских аналогов: IntelliJ IDEA, AndroidStudio, OpenIDE, GigaIDE, PyCharm, GoLand, Rider и WebStorm. Плагин Veai включает в себя AI-агента и неагентскую функциональность такую как, генерация тестов по конкретному исполнению.

Рассмотрим, как работает AI-агент Veai. Агент получает от пользователя задачу разработки, тестирования или отладки и затем с помощью большой языковой модели (LLM) генерирует вызовы инструментов взаимодействия с проектом, необходимые для сбора контекста и решения задачи. Полученные результаты работы инструментов дописываются в конец запроса к LLM, называемого «промптом», после чего LLM снова генерирует вызовы инструментов взаимодействия с проектом. Данный цикл повторяется, пока LLM не сгенерирует ответ содержащий не вызовы инструментов, а отчёт о проделанной работе или уточняющий вопрос или не сработает механизм прерывания агентского цикла. Упрощённая схема работы показана на рисунке 2.

Такая схема взаимодействия с LLM является весьма стандартной [20], особенностью Veai является тесная интеграция агента с IDE, выпускаемыми компанией JetBrains, в частности с IntelliJ IDEA. Эта интеграция заключается в обогащении промпта контекстом и состоянием проекта и, самое главное, предоставлением агенту ряда инструментов, позволяющих использовать возможности IDE, например:

- инструмент просмотра структуры файла позволяет агенту получить данные о содержимом файла (классах, методах, XML-секциях и т. д.), не расходуя контекст на полное прочтение файла;

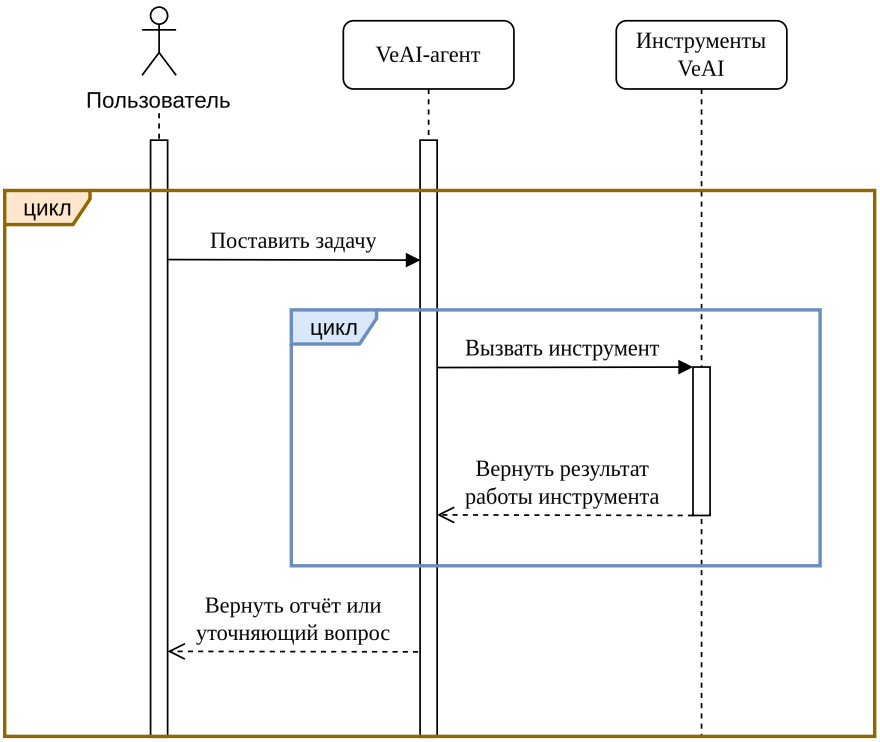

Рис. 2: Упрощённая схема работы Veai агента.

- инструмент поиска и чтения файлов позволяет агенту работать не только с файлами самого проекта, но и с файлами библиотек, в том числе в тех случаях, когда исходный код библиотек не доступен и требуется декомпиляция;
- инструмент редактирования файла сразу же сообщает агенту о допущенных ошибках компиляции, обнаруженных IDE, без необходимости пересобирать весь проект или весь модуль;
- инструменты запуска команд и задач системы сборки, позволяют агенту выполнять команды и задачи Gradle и Maven с учётом настроек IDE, таких как настройки версии SDK;
- инструмент запуска тестов позволяет агенту запускать тесты с теми же параметрами, с которыми их запускает IDE, собирать структурированные сжатые результаты запуска тестов: данные журналирования (логи), исключения и данные о покрытии кода тестами.

Перечисленные инструменты уже позволяют AI-агенту Veai часто

успешно решать задачу исправления ошибок реализации, когда достаточно исправить тот метод или класс, из которого было выброшено исключение. Тем не менее на момент начала данной работы оставалось затрудненным исправление ошибок в тех случаях, когда содержащий ошибку метод или класс не упоминается в трассе стека (англ. stacktrace) исключения, особенно когда содержащий ошибку метод отделён от непрошедшего (падающего) теста десятками кадров стека вызовов, в том числе в приложениях, использующих рефлексию. Данное ограничение связано с тем, что AI-агент Veai не имеет доступа к детальным данным времени выполнения и при генерации исправлений, основывается лишь на логах падающих тестов и собранном с помощью вызова инструментов анализа проекта контексте.

2.2 Способы представления данных времени выполнения

Исправление ошибок реализации с помощью AI-агентов является популярной темой исследований [18, 35, 31, 11, 13, 25, 2]. В данном разделе проводится обзор того, какого рода данные времени выполнения пробовали передавать в AI-агентов в предыдущих работах по теме, как эти данные представлялись, и насколько такие представления могут быть полезны при исправлении ошибок, спрятанных за десятками кадров стека вызовов (аналогичных примеру рассмотренному во введении настоящей работы, см. рисунок 1).

Перед переходом к рассмотрению конкретных работ по теме стоит отметить важную особенность данной области исследований: большие языковые модели (LLM) стремительно развиваются в последние годы. Существенная доля задач в области разработки ПО, которые не решались моделями, доступными в 2024 году, успешно решаются современными языковыми моделями [27]. Научные статьи публикуются с существенной задержкой во времени, поэтому при рассмотрении даже недавних исследований надо учитывать не только их результаты, полученные на далеко не самых современных моделях, но и то, насколько

используемые в этих исследованиях подходы обобщаемы для решения задач, всё ещё вызывающих трудности у современных LLM, в частности задач, требующих анализа на много методов вглубину.

2.2.1 Данные журналирования (логи) непроходящих тестов

Простейшими данными времени выполнения, которые нельзя не упомянуть, являются данные журналирования (логи) падающих тестов. Ранние работы, посвящённые исправлению ошибок (дефектов) реализации с помощью больших языковых моделей используют логи в качестве единственных данных времени выполнения [2]. Однако, как уже было сказано в подразделе 2.1, логи падающих тестов уже доступны AI-агенту Veai, следовательно для повышения точности исправления ошибок AI-агентом Veai необходимо рассмотреть другие подходы.

2.2.2 Трассы выполнения

Ещё одним видом данных времени выполнения, которые могут помочь AI-агенту более точно исправлять ошибки реализации являются трассы выполнения, показывающие путь выполнения и отладочную информацию в различных точках этого пути. Данные такого вида по разному представлялись в нескольких предшествующих работах [18, 35, 31].

В работе [18] в контекст языковых моделей включается код тестируемой функции и при этом в конец каждой выполненной строки кода в этой функции добавляется комментарий, показывающий значение всех переменных в момент первого и последнего выполнений этой строки кода. Данный подход позволяет языковым моделям более точно отлаживать отдельные функции, однако в масштабах отдельных функций языковые модели уже показывают выдающиеся результаты и без использования трасс выполнения [17, 8], а при использовании данного подхода для межпроцедурной отладки теряется информация о порядке вызова функций, кроме этого включение в промпт кода каждого метода, выполненного во время выполнения теста, при отладке реаль-

ных приложений рискует переполнить эффективное контекстное окно языковой модели.[2]

В исследовании [31] предлагается подход, похожий на подход из [18], но на этот раз также исследуется подход, при котором значения переменных записываются в контекст языковой модели не в виде комментариев внутри исходного кода, а в виде отдельной секции промпта. В [31], как и в [18], трассы строятся на уровне отдельных строк кода, что также делает этот подход малоприменимым для глубокой межпроцедурной отладки: результаты исследования показывают, что успешность исправления ошибок реализации отрицательно коррелирует с длиной трассы выполнения и что наиболее стабильные результаты получаются при предварительном сжатии трассы выполнения с помощью дополнительного запроса к языковой модели [31].

Наконец, в статье [35] рассматривается включение в запрос к LLM трасс различных уровней гранулярности: трасс на уровне отдельных строк кода, трасс на уровне базовых блоков (англ. basic blocks) или трасс на уровне функций. Для каждого вида трассы наблюдается увеличение количество успешных исправлений ошибок в тестируемой программе, наилучшие результаты получаются для трасс выполнения на уровне базовых блоков. В контексте рассматриваемой в данной работе задачи глубокой межпроцедурной отладки наиболее перспективными выглядят низкие уровни гранулярности трасс выполнения, такие как гранулярность на уровне функций и, возможно, гранулярность на уровне базовых блоков. К сожалению, переиспользовать разработанный в [35] инструментарий сбора трасс в рамках данной работы не представляется возможным, так как этот инструментарий предназначен для программ на языке Python, а настоящая работа фокусируется на JVM-языках.

[2]Эффективное контекстное окно LLM — это часть контекстного окна, которую LLM может эффективно использовать для решения нетривиальных задач. Бенчмарки показывают, что у современных LLM эффективное контекстное окно существенно меньше, чем всё контекстное окно [22].

2.2.3 Интерактивно собираемые данные

Ряд исследований [11, 13, 25] вместо полного сбора трасс выполнения фокусируется на сборе данных времени выполнения в некоторых контрольных точках. В одних работах [11, 13] эти контрольные точки определяются самой большой языковой моделью в виде условных точек останова (англ. conditional breakpoint), в то время, как в [25] пользователю инструмента самому предлагается поставить точку остановки в интересном месте отлаживаемой программы и задать языковой модели вопрос про текущее состояние времени выполнения и то, как оно было достигнуто. При этом в работе [11] большая языковая модель сама определяет выражение, значение которого будет вычислено в точке остановки, а в исследовании [13] языковая модель не только может вычислять значения выражений в точке остановки, но и произвольным образом изменять состояние программы.

В контексте задачи настоящей работы, а именно в контексте расширения возможностей AI-агента Veai необходимо учесть, что AI-агент Veai уже в состоянии собирать данные времени выполнения интерактивно путём добавления временных инструкций логирования с помощью инструмента редактирования файлов и повторных запуска тестов после добавления этих отладочных инструкций.

2.2.4 Выбор способа представления данных времени выполнения

Итак, два из трёх рассмотренных видов данных времени выполнения уже в некоторой степени доступны AI-агенту Veai: данные журналирования (логи) полностью доступны AI-агенту Veai, а интерактивно собираемые данные агент может получить, воспользовавшись инструментами редактирования файлов и запуска тестов. У оставшегося подхода, а именно у сбора трасс выполнения можно выделить три уровня гранулярности: уровень строк, уровень базовых блоков (basic blocks) и уровень функций. При этом для трасс уровня отдельных строк характерен наиболее высокий риск переполнить эффективный размер контек-

Таблица 1: Результаты сравнения различных данных времени выполнения в контексте задачи повышения точности исправления ошибок реализации AI-агентом Veai.

Вид данных времени выполнения		**Низкий риск переполнения эффективного контекстного окна**	**Может повысить качество работы Veai**[3]
Данные журналирования (логи)		$+$[4]	$-$
Интерактивно собираемые данные		$\pm$	$\pm$
Трассы выполнения	Детальность уровня строк кода	$-$	$+$
	Детальность уровня базовых блоков	$\mp$	
	Детальность уровня функций	$\pm$	

ста. Компактно результаты сравнения рассмотренных видов данных времени выполнения представлено в таблице 1.

Учитывая, что предыдущие исследования показывают, что в различных случаях полезны разные данные времени выполнения [13] и единственным видом данных времени выполнения, полностью недоступным AI-агенту Veai, являются трассы выполнения, в рамках настоящей работы было решено поддержать в Veai использование именно трасс выполнения. В частности, из-за соображений рисков переполнения контекста было решено использовать трассы уровня функций (см. предпоследний столбец таблицы 1).

Стоит отметить, что то, что в предыдущих работах удалось найти перспективную идею представления данных времени выполнения в виде трассы выполнения на уровне функций вовсе не означает, что задача о представлении этой трассы в контексте языковой модели является решённой. Существенной сложностью остаётся сжатие реальных трасс выполнения, часто состоящих из десятков или сотен тысяч вызовов и уходящих на десятки методов в глубину, до разумных размеров. В рассмотренных в обзоре работах [18, 35, 31] данная проблема

[3]Не покрыто исходными возможностями Veai.

[4]При использовании логов риск переполнения контекста низкий, так как в Veai реализован механизм сжатия логов выполнения тестов.

сжатия межпроцедурных трасс выполнения не решалась, так как эти работы ориентировались на бенчмарки [10, 1, 24, 33, 26, 23], состоящие из небольших программ, каждая из которых содержит лишь несколько функций.

2.3 JVM-технологии сбора трасс выполнения

Как уже было сказано в подразделе 2.2, в настоящей работе не получится переиспользовать сборщик трасс выполнения уровня функций, используемый в [35], так как настоящая работа в отличие от [35] ориентирована не на язык Python, а на JVM-языки, то есть языки, которые исторически имеют наилучшую поддержку в Veai. К счастью, в мире JVM существует множество инструментов и библиотек, позволяющих реализовать сбор трасс выполнения, таких как OpenTelemetry [7], Spring AOP [3], Java Debug Interface (JDI) [21], AspectJ [9], JacoDB [14], Soot [29], ByteBuddy [34], Javassist [6] и ASM [19]. Сравнение перечисленных технологий приведено в таблице 2.

При выборе технологии надо было учесть, что реальные приложения на JVM-языках обычно используют библиотеки и при наивном включении в трассу всех вызовов функций большая часть трассы будет описывать внутренности работы библиотек. Кроме того, сбор всех вызовов методов внутри всех библиотек рискует существенно замедлить работу приложения. В связи с этим необходимо было выбрать инструмент предоставляющий высокоуровневый программный интерфейс (API) для инструментацию только тех вызовов методов, которые происходят из кода самого приложения, а не из библиотек.

Среди рассмотренных технологий все, кроме одной, либо не поддерживают такую инструментацию либо поддерживают её лишь на низком сложном в использовании уровне. В таблице 2 единственным инструментом, предоставляющим высокоуровневый API для инструментации на стороне вызывающего метода (англ. caller side instrumentation) является AspectJ. Именно этот инструмент и было решено использовать в данной работе.

Таблица 2: Сравнение JVM-технологий, которые можно использовать для сбора трасс выполнения.

Технология	**Применимость к вызовам любых методов**	**Высокоуровневый API для инструментации на стороне вызываемого метода**	**Высокоуровневый API для инструментации на стороне вызывающего метода**
OpenTelemetry[5]	−[6]	+	−
Spring AOP	−[7]	+	−
JDI	+	−	−
AspectJ	+	+	+
JacoDB	+	−	−
Soot	+	−	−
ByteBuddy	+	+	−
Javassist	+	+	±
ASM	+	−	−

[5]Если использовать только OTel Java-агента без дополнительной (самописной) инструментации.

[6]Только вызовы некоторых библиотечных методов.

[7]Только вызовы методов объектов, управляемых фреймворком Spring.

3 Архитектура

В данной главе описывается и обосновывается архитектура решения, позволившая встроить в AI-агента Veai сбор и фильтрацию данных времени выполнения. Базовая архитектура AI-агента Veai была рассмотрена в разделе 2.1, поэтому далее рассматриваются только новые и доработанные компоненты, связи между этими компонентами и новые сценарии работы.

При проектировании архитектуры решения учитывались следующие ограничения.

1. Набор доступных AI-агенту Veai инструментов не должен расширяться без резкой необходимости (чрезмерное расширение набора инструментов может отрицательно повлиять на точность выбора инструментов агентом).

2. Решение должно сохранять совместимость со всеми поддержанными в Veai системами сборки (Gradle, Maven, IntelliJ Build System) и тестовыми фреймворками (JUnit 5, JUnit 4, TestNG).

3. Предоставляемые AI-агенту Veai данные времени выполнения должны помещаться в эффективное контекстное окно языковой модели (может понадобиться реализация различных фильтров данных времени выполнения, учитывающих глубину вложенности данных, повторы данных и другие факторы).

4. Пользователь должен видеть прогресс анализа данных времени выполнения.

Учитывая ограничения 1 и 2, было решено встроить сбор и фильтрацию данных времени выполнения в существующий инструмент запуска тестов, а не выносить их в отдельный инструмент AI-агента. Такой подход позволил переиспользовать существующие интеграции с системами сборки и тестовыми фреймворками, а также предоставить AI-агенту единый инструмент как для получения базовых результатов запуска

тестов, так и для получения отфильтрованных данных времени выполнения.

Учитывая ограничение 3 и выводы из обзора (см. раздел 2.2.4), было решено в качестве данных времени выполнения использовать трассу выполнения уровня функций, каждый элемент которой характеризуется сведениями о вызываемом методе, месте вызова, аргументах, результате или исключении и времени выполнения. При этом, чтобы сделать возможной реализацию широкого круга фильтров данных времени выполнения, трасса выполнения представлялась не в виде плоского списка вызовов методов, а в виде дерева вызовов, сохраняющего информацию о вложенности вызовов. В таком представлении, если метод `inner()` вызывается в ходе выполнения метода `outer()`, то соответствующий вызов метода `outer()` является родителем вызова метода `inner()`.

Для соблюдения ограничения 4 было решено, что запросы к большим языковым моделям, используемые для анализа данных времени выполнения, должны показываться пользователю в потоковом режиме.

3.1 Роли и связи новых и доработанных компонентов

Ориентируясь на описанные выше решения, были выделены компоненты, представленные на рисунке 3.

Компонент `veai-runtime` отвечает за мониторинг Java-процессов, запущенных Veai. В данной работе в компоненте `veai-runtime` были созданы подкомпоненты `call-tree-dumper` и `call-tree-model`, использующие библиотеку AspectJ для реализации Java-агента и отвечающие соответственно за сбор и представление «сырой» трассы выполнения.

Компонент `veai-test-runner` отвечает за запуск тестов в отдельном Java-процессе. В рамках данной работы этот компонент был интегрирован с реализованным в компоненте `veai-runtime` сборщиком трассы выполнения: при запуске теста компонент подключает Java-агент, получает от него «сырую» трассу выполнения и передаёт её в дальнейшую обработку.

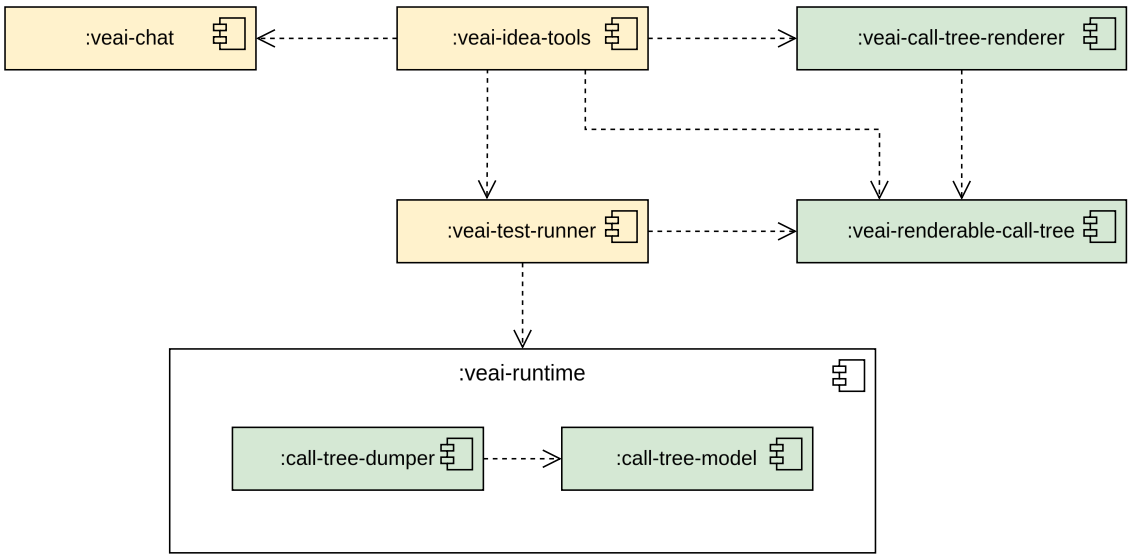

Рис. 3: Диаграмма компонентов, созданных и доработанных в ходе работы (зелёным цветом показаны новые компоненты, жёлтым цветом показаны доработанные компоненты).

Компонент `veai-renderable-call-tree` был создан в ходе данной работы и задаёт представление трассы выполнения, удобное для постобработки и рендеринга. Это представление отличается от «сырого» представления в компоненте `call-tree-model` тем, что содержит дополнительные свойства, с помощью которых более ранние фильтры трассы выполнения могут влиять на поведение более поздних фильтров. Например, фильтр, отвечающий за сохранение репрезентативных вызовов (см. раздел 4.2.3), использует эти дополнительные свойства, чтобы защитить часть вызовов от удаления при применении последующих фильтров.

Компонент `veai-call-tree-renderer` был создан в ходе данной работы и отвечает за постобработку и рендеринг трассы выполнения. Постобработка организована как конвейер фильтров с единым интерфейсом. Такой подход снижает когнитивную нагрузку при разработке и сопровождении решения: каждый фильтр отвечает за одну трансформацию, а не за весь процесс подготовки трассы целиком. Конвейер включает фильтры для присоединения асинхронных вызовов, фокусировки на релевантных участках трассы, сохранения репрезентативных вызовов, удаления локальных повторов, сжатия неинформативных иден-

тификаторов объектов, ограничения размера трассы и суммаризации трассы выполнения с помощью большой языковой модели. Подробнее эти фильтры описаны в разделе 4.2.

Компонент `veai-idea-tools` отвечает за реализацию инструментов агента Veai для взаимодействия со средой разработки IntelliJ IDEA. В данной работе реализованный в этом компоненте инструмент запуска тестов был обновлён так, чтобы агент мог запросить сбор трассы выполнения и получить в результате запуска теста суммаризацию трассы и путь к файлу со сжатой трассой выполнения.

Компонент `veai-chat` отвечает за высокоуровневую работу чата пользователя с агентом Veai. В данной работе был произведён рефакторинг этого компонента, позволивший инструментам AI-агента Veai обновлять показываемый пользователю результат в произвольное время, а не только в конце работы инструмента. В частности, данное расширение позволило инструменту запуска тестов показать пользователю прогресс анализа трассы выполнения в потоковом режиме.

3.2 Последовательность работы

При вызове Veai-агентом инструмента запуска тестов, реализованного в компоненте `veai-idea-tools`, выполняется следующая последовательность действий, показанная на рисунке 4.

1. Компонент `veai-test-runner` запускает отдельный JVM-процесс тестируемого проекта и подключает Java-агент, реализованный в компоненте `call-tree-dumper`.

2. Java-агент формирует «сырую» трассу выполнения в формате `call-tree-model`.

3. Компонент `veai-test-runner` преобразует «сырую» трассу выполнения вызовов в абстракции компонента `veai-renderable-call-tree`.

4. Компонент `veai-call-tree-renderer` применяет конвейер фильтров и рендерит компактное представление трассы выполнения.

5. Реализация инструмента запуска тестов в компоненте `veai-idea-tools` отслеживает процесс суммаризации трассы выполнения и в потоковом режиме передает результаты в компонент `veai-chat`, который теперь позволяет инкрементально обновлять результат работы инструментов AI-агента Veai.

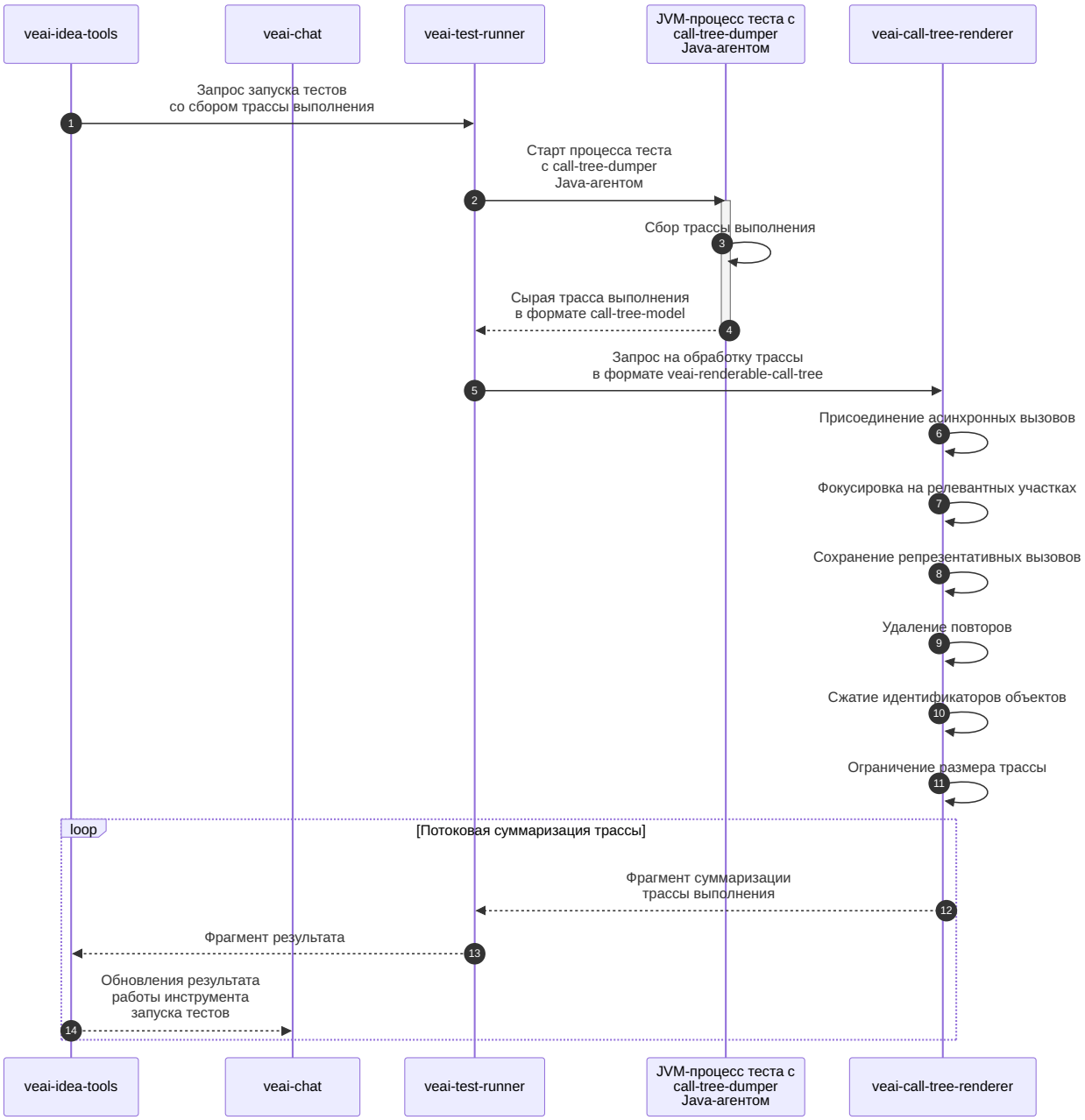

Рис. 4: Диаграмма последовательности выполнения инструмента запуска тестов со сбором и анализом трассы выполнения.

4 Особенности реализации

В данной главе описываются ключевые особенности реализации подсистемы сбора и фильтрации трасс выполнения, архитектура которой представлена в разделе 3.

4.1 Сбор трассы выполнения

Для сбора трассы выполнения в виде дерева вызовов реализован механизм, автоматически запускающий тесты на JVM-языках с разработанным в рамках данной работы Java-агентом на основе AspectJ. Механизм переиспользует возможности среды разработки IntelliJ IDEA и потому работает с любыми поддерживаемыми средой разработки тестами на JVM-языках вне зависимости от фреймворка тестирования и системы сборки.

При реализации сборщика трасс выполнения учитывались следующие два соображения.

1. Сбор трассы не должен требовать инструментации всех внутренних классов используемых библиотек — это стандартное требование к подобным Java-агентам [32], мотивированное тем, что Java-приложения нередко используют крайне объёмные библиотеки, такие как Spring [4], полная инструментация которых непрактична.

2. В трассе должны быть представлены вызовы библиотечных методов, выполняемые **напрямую** из кода приложения (см. пример на рисунке 5).

Совместное соблюдение обоих соображений нетривиально. Рассмотрим пример из рисунка 5: метод приложения `appMethod()` получает значения `"val1"` и `"val2"` от `appMethod1()` и `appMethod2()` соответственно, после чего вызывает библиотечный метод `heavyLibraryMethod()`, ошибочно передавая ему аргументы в обратном порядке (`"val2"`, `"val1"`). Причина ошибки находится в коде приложения — в строке вызова `heavyLibraryMethod()`, поэтому трасса должна содержать этот вызов с

фактическим порядком аргументов. Для построения такой трассы возможны три способа инструментации.

Инструментация всех методов. В трассу попадают вызовы как методов приложения (`appMethod1()`, `appMethod2()`), так и `heavyLibraryMethod()` со всеми его внутренними вызовами (`internalLibMethod1()`, `internalLibMethod2()` и далее — в реальных библиотеках это могут быть сотни тысяч вызовов). Запись `heavyLibraryMethod("val2", "val1")` в трассе присутствует, и ошибка локализуется, однако трансформация всего библиотечного кода противоречит соображению 1 и делает подход непрактичным.

Инструментация только методов приложения с регистрацией событий на стороне вызываемого метода (callee-side инструментация). При входе в каждый метод приложения и при выходе из него регистрируются события трассы, библиотечные методы не инструментируются. Трасса компактна и не требует трансформации библиотек, но содержит только вызовы `appMethod()`, `appMethod1()` и `appMethod2()`, а сам факт вызова `heavyLibraryMethod()` и переданные аргументы отсутствуют. Ошибка не локализуется, что не соответствует соображению 2.

Инструментация только методов приложения с регистрацией событий на стороне вызывающего метода (caller-side инструментация). Дополнительно к событиям входа в методы приложения и выхода из методов приложения регистрируются события вызова любых методов (включая библиотечные), выполняемые **непосредственно из методов приложения**. В трассу попадает `heavyLibraryMethod("val2", "val1")` с фактическими аргументами, при этом внутренности `heavyLibraryMethod()` не инструментируются. Данный подход, проиллюстрированный в правой нижней части рисунка 5, удовлетворяет обоим соображениям и используется в настоящей работе.

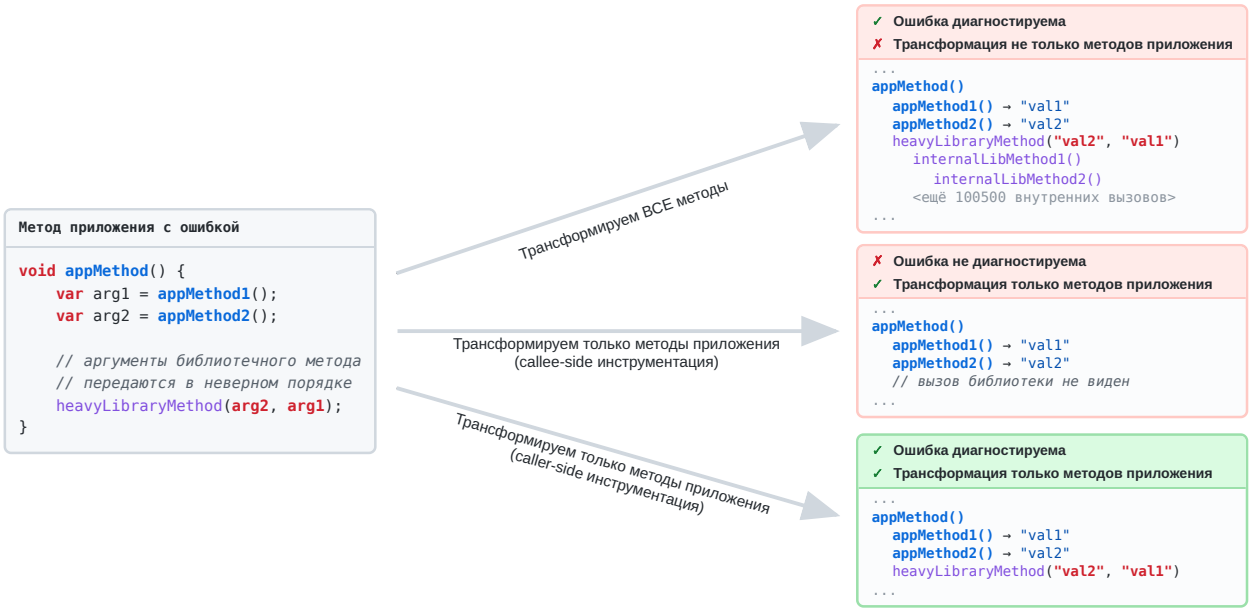

Рис. 5: Пример ошибочного вызова библиотечного метода и влияние способа инструментации на структуру трассы выполнения.

4.2 Фильтрация трассы выполнения

«Сырая» трасса выполнения, возвращаемая Java-агентом, может существенно превосходить размер эффективного контекстного окна современных языковых моделей и не отражает связи между асинхронными вызовами. Поэтому к «сырой» трассе применяется разобранный в данном подразделе конвейер шагов постобработки.

4.2.1 Присоединение асинхронных вызовов

В «сырой» трассе выполнения асинхронные вызовы представлены отдельными верхнеуровневыми элементами списка вызовов, то есть неизвестно, откуда эти асинхронные вызовы совершаются. На первом шаге постобработки трассы выполнения асинхронные вызовы ассоциируются с наиболее подходящими местами вызовов на основе записанной Java-агентом информации о времени начала и конца вызовов (см. пример на рисунке 6).

Этот шаг постобработки защищает содержательные асинхронные вызовы и всё, что внутри них происходит, от удаления на последующих шагах постобработки трассы выполнения (например, на рисунке 6 при-

(а) Трасса выполнения до присоединения асинхронных вызовов

```
...
MyControllerTest.exampleTest() → thrown AssertionError("Expected 200...")
  WebTestClient.get() → RequestHeadersUriSpec@19af24e1(...)
    RequestHeadersUriSpec.uri("/test") → RequestHeadersUriSpec@19af24e1(...)
    RequestHeadersUriSpec.exchange() → DefaultResponseSpec@a7511c8b(status=500, ...)
      DefaultResponseSpec@a7511c8b.expectStatus() → StatusAssertions@b2e631d7
        StatusAssertions@b2e631d7.isOk() → thrown AssertionError(...)
MyController.get() → ResponseEntity@7b6fa720(status=500, ...)
  <участок трассы с первопричиной ошибки>
...
```

(б) Трасса выполнения после присоединения асинхронных вызовов

```
...
MyControllerTest.exampleTest() → thrown AssertionError("Expected 200...")
  WebTestClient.get() → RequestHeadersUriSpec@19af24e1(...)
    RequestHeadersUriSpec.uri("/test") → RequestHeadersUriSpec@19af24e1(...)
    RequestHeadersUriSpec.exchange() → DefaultResponseSpec@a7511c8b(status=500, ...)
      (async in thread worker-3) MyController.get() → ResponseEntity@7b6fa720(status=500, ...)
        <участок трассы с первопричиной ошибки>
    DefaultResponseSpec@a7511c8b.expectStatus() → StatusAssertions@b2e631d7
    StatusAssertions@b2e631d7.isOk() → thrown AssertionError(...)
...
```

Рис. 6: Пример присоединения асинхронного вызова метода `MyController.get()`, выполняемого из метода `RequestHeadersUriSpec.exchange()`.

соединение асинхронного вызова сохраняет участок, отмеченный текстом `<участок трассы с первопричиной ошибки>`, от удаления на шаге из подраздела 4.2.2).

4.2.2 Фокусировка на релевантных участках трассы

Следующим шагом работы конвейера является фокусировка на релевантных участках трассы выполнения. На этом шаге удаляются попавшие в трассу вызовы, происходящие за пределами «интересуемого метода». Интересуемый метод определяется самим AI-агентом Veai, обычно это тестовый метод, но иногда — один из методов, выполняющий инициализацию релевантной части системы до выполнения теста.

4.2.3 Сохранение репрезентативных вызовов

Для каждой достаточно популярной пары вызываемого метода и результата вызова выделяется представитель. Представители защища-

ются от удаления на последующих шагах сжатия, что предотвращает полное исчезновение целых групп вызовов.

4.2.4 Удаление локальных повторов

На шаге удаления локальных повторов сжимаются группы смежных вызовов одних и тех же методов из одних и тех же мест вызова с одним и тем же типом результата. Такие группы возникают в циклах при выполнении большого количества однотипных операций, их дедупликация освобождает место для более информативных участков трассы выполнения на последующих шагах сжатия.

4.2.5 Сжатие неинформативных идентификаторов объектов

В строковых представлениях объектов JVM часто встречаются идентификаторы вида `MyClass@7b6fa720`. В таких идентификаторах часть после символа «`@`» малоинформативна, но занимает много токенов[8], поэтому такие длинные идентификаторы заменяются на короткие псевдонимы: например, все упоминания `MyClass@7b6fa720` могут замениться на `MyClass@42`.

4.2.6 Ограничение размера трассы с распределением бюджета

Даже после дедупликации повторов трасса выполнения может оставаться слишком большой. Поэтому размер трассы ограничивается с помощью рекурсивного распределения бюджета токенов между оставшимися в трассе вызовами методов. Сначала весь бюджет распределяется между верхнеуровневыми вызовами «интересуемого метода», выбранного AI-агентом Veai. Затем рекурсивно каждый вызов, получивший часть бюджета, распределяет свою часть бюджета между происходящими внутри него вызовами других методов. При этом больше бюджета

[8]Современные языковые модели используют токенизацию на основе алгоритмов семейства BPE [28], которые объединяют в один токен часто встречающиеся в обучающей выборке последовательности символов. Каждое конкретное шестнадцатеричное значение вроде `7b6fa720` в таких выборках встречается редко, поэтому разбивается на токены по 1–2 символа.

получают те вызовы, в которых выбрасываются или ловятся исключения. Участки трассы, для которых бюджета не хватило, заменяются на текст, поясняющий, что часть трассы не показана.

4.2.7 Суммаризация трассы выполнения с помощью большой языковой модели

После выполнения всех механических шагов фильтрации трассы выполнения, сжатая трасса суммаризуется большой языковой моделью и результат суммаризации возвращается AI-агенту Veai в ответе инструмента запуска тестов. Дополнительно AI-агенту передаётся путь к файлу со сжатой трассой, что позволяет агенту при необходимости найти и прочитать релевантные участки трассы выполнения.

5 Экспериментальное исследование

В данной главе описывается методология и результаты экспериментального исследования, направленного на оценку влияния использования трасс выполнения на качество генерируемых Veai исправлений дефектов в Java-приложениях.

5.1 Исследовательские вопросы

Перед проведением замеров были поставлены следующие исследовательские вопросы.

RQ1. Как использование трасс выполнения влияет на процент генерируемых Veai исправлений, которые проходят автотесты?

RQ2. Как использование трасс выполнения влияет на процент генерируемых Veai исправлений, которые корректно устраняют первопричину дефекта?

5.2 Метрики

Для ответа на первый вопрос измерялась следующая метрика.

M1 (Прохождение тестов). Процент сгенерированных AI-агентом исправлений, проходящих автотесты из проекта.

Для ответа на второй вопрос использовались следующие две прокси-метрики[9].

M2 (Прохождение тестов и изменения ожидаемого файла). Процент сгенерированных AI-агентом исправлений, которые соответствуют требованиям метрики M1 и изменяют «ожидаемый файл» (файл, который оригинальные разработчики соответствующего Java-проекта отредактировали для исправления рассматриваемого дефекта).

[9]Прокси-метрика (замещающая метрика) — это косвенный показатель, который сильно коррелирует с главной, трудноизмеримой целевой метрикой.

M3 (Эквивалентность эталонному исправлению). Процент сгенерированных AI-агентом исправлений, которые соответствуют требованиям метрик M1 и M2 и при использовании запроса из Листинга 1 признаются языковой моделью GPT-5.4 (xhigh) семантически корректными, то есть устраняющими ту же первопричину дефекта, что и эталонное исправление от разработчиков соответствующего проекта.

Прокси-метрики M2 и M3 имеют различные достоинства и недостатки. Метрика M2 строго определена, однако эта метрика считает корректными исправления, которые хотя и редактируют ожидаемый файл, но исправляют поведение только в покрытых тестами сценариях. Метрика M3 наоборот не ограничивается формальными тестами и оценивает, устраняет ли сгенерированное исправление первопричину дефекта, но эта метрика вычисляется не с помощью небольшой процедуры, а с помощью большой языковой модели, то есть использует подход «большая языковая модель, как судья» (англ. LLM as a judge).

5.3 Условия эксперимента

Чтобы исключить зависимость результатов от особенностей конкретной языковой модели, для проведения эксперимента использовались три различные большие языковые модели, созданные различными компаниями. Проприетарные модели Claude Opus 4.6 (adaptive) и GPT-5.4 (xhigh) использовались по API, модель с открытыми весами DeepSeek V3.2 (AWQ) запускалась на сервере, оснащённым четырьмя графическими процессорами (GPU) модели NVIDIA H200. Для выполнения совершаемых AI-агентом Veai вызовов инструментов использовалась рабочая станция с процессором 13th Gen Intel(R) Core(TM) i7-13700H и 32 ГБ оперативной памяти.

Для проведения замеров использовался набор данных из 43 нетривиальных дефектов в 24 Java-проектах из набора данных BugSwarm [5]. Чтобы оценить вариативность результатов, для каждой пары дефекта

Листинг 1. Запрос к языковой модели, использовавшийся для оценки семантической эквивалентности сгенерированного исправления и эталонного исправления в рамках метрики M3. Перед отправкой запроса в языковую модель подстановочный параметр `{GOLD_DIFF}` заменялся на эталонное исправление от разработчиков проекта, параметр `{GENERATED_DIFF}` — на сгенерированное Veai исправление, а параметр `{BUGGY_FILE_CONTENT}` — на исходное содержимое файла, который был отредактирован эталонным исправлением.

```
<GoldDiff>
{GOLD_DIFF}
</GoldDiff>

<GeneratedDiff>
{GENERATED_DIFF}
</GeneratedDiff>

<BuggyFileContent>
{BUGGY_FILE_CONTENT}
</BuggyFileContent>

<Instructions>
You are evaluating a generated diff for a Java bug-fix task. The gold
diff shows the known correct fix. The generated diff is what an AI agent
produced. Tests pass with the generated diff (formal verification
succeeded).

Classify the generated diff as either:
- SEMANTICALLY_CORRECT: The generated diff produces a correct fix that
  addresses the same root problem as the gold diff. The generated fix
  can be broader than the gold fix.
- SEMANTICALLY_INCORRECT: The generated diff makes tests pass but the
  fix is wrong: the fix will break under scenarios not covered by the
  current tests.

Reason step by step:
1. What does the gold diff fix and how?
2. What does the generated diff do?
3. Is the generated diff semantically correct?

Response format is STRICT:
<Reasoning>[3-6 sentences of analysis]</Reasoning>
<FixValidity>[SEMANTICALLY_CORRECT|SEMANTICALLY_INCORRECT]</FixValidity>

</Instructions>
```

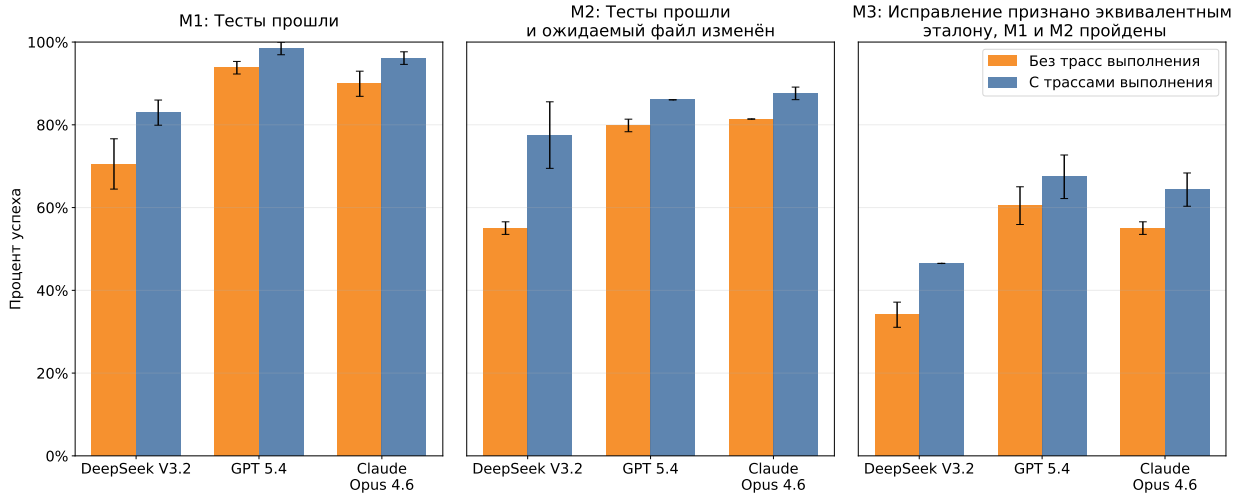

Рис. 7: Процент успешных исправлений по метрикам M1, M2, M3. Планки погрешностей соответствуют 95 % доверительным интервалам.

и модели проводилось три замера с использованием трасс выполнения и три замера без использования трасс выполнения.

Время на исправление одного дефекта было ограничено 30 минутами. Отсчёт времени начинался после полной индексации средой разработки IntelliJ IDEA и плагином Veai проекта и выполнения команды `gradle assemble` для Gradle-проектов и команды `mvn -T1C -DskipTests -Dspotbugs.skip=true -Dpmd.skip=true -Dcheckstyle.skip=true install` для Maven-проектов.

В процессе экспериментальных запусков AI-агенту Veai передавалась задача в виде запроса «Run `{FAILED_TEST}` and fix the bug in the implementation», где вместо `{FAILED_TEST}` подставлялось имя непроходящего теста.

5.4 Результаты

В данном подразделе на основе результатов $2 \times 3 \times 43 \times 3 = 774$ запусков[10] AI-агента Veai даются ответы на поставленные исследовательские вопросы. Полученные в результате замеров значения метрик M1, M2 и M3 показаны на рисунке 7.

[10] Агент запускался в двух режимах: режиме с трассой вызовов и режиме без трассы вызовов, — для каждой из трёх языковых моделей и каждого из 43 дефектов по три раза.

Ответ на RQ1. Использование трасс выполнения увеличивает процент проходящих автотесты исправлений (метрика M1 на рисунке 7) на 4,7–12,4 процентных пунктов в зависимости от языковой модели.

Ответ на RQ2. Рост прокси-метрик M2 и M3, показанный на рисунке 7, свидетельствует о том, что трассы выполнения позволяют AI-агенту Veai чаще корректно устранять первопричины дефектов в Java-приложениях. В частности, благодаря трассам выполнения, процент исправлений, проходящих автотесты и сделанных в ожидаемом файле, увеличивается на 6,2–22,5 процентных пунктов в зависимости от языковой модели, а процент исправлений, которые помимо прохождения формальных проверок ещё и признаются независимой языковой моделью семантически эквивалентными эталонному исправлению, увеличивается на 7,0–12,4 процентных пунктов.

Заключение

В ходе работы были достигнуты следующие результаты.

1. В результате сравнительного анализа трёх подходов и девяти технологий для сбора данных времени выполнения установлено, что наиболее подходящим для использования в Veai решением является создание подсистемы сбора и фильтрации трассы выполнения с использованием библиотеки AspectJ.

2. Спроектирована архитектура решения, которая позволила, используя библиотеку AspectJ, реализовать в AI-агенте Veai подсистему сбора и многошаговой фильтрации трассы выполнения, а также позволила показать пользователям инструмента прогресс анализа трассы выполнения.

3. Спроектированное решение реализовано и встроено в AI-агента Veai.

4. На сорока трёх реальных ошибках, выявленных автотестами в двадцати четырёх проектах из набора данных BugSwarm, экспериментально установлено, что для языковых моделей Claude Opus 4.6, GPT 5.4 и DeepSeek V3.2 предложенное решение увеличивает:

   - долю исправлений, проходящих автотесты, на 4,7–12,4 процентных пунктов в зависимости от модели;
   - долю исправлений, проходящих автотесты и сделанных в ожидаемом файле, на 6,2–22,5 процентных пунктов в зависимости от модели;
   - долю исправлений, признанных эквивалентными эталонным исправлениям[11], на 7,0–12,4 процентных пунктов в зависимости от модели.

[11] Для оценки семантической корректности исправления использовалась комбинация прохождения автотестов, попадания исправлением в ожидаемый файл и подхода «языковая модель, как судья» (англ. LLM as a judge), где языковой модели передавались содержимое исправляемого файла, эталонное исправление и сгенерированное исправление.

5. Результаты работы были представлены на конференции JPoint 2026 [16].

Список литературы

[1] ASSERT Lab, KTH Royal Institute of Technology. HumanEval-Java. — GitHub repository. — 2023. — URL: https://github.com/ASSERT-KTH/human-eval-java (дата обращения: 5 января 2026 г.).

[2] Bouzenia Islem, Devanbu Premkumar T., Pradel Michael. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair // Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. — ICSE '25. — IEEE, 2025. — P. 2188–2200. — URL: https://doi.org/10.1109/ICSE55347.2025.00157 (дата обращения: 5 января 2026 г.).

[3] Broadcom Inc. Spring AOP. — 2025. — URL: https://docs.spring.io/spring-framework/reference/core/aop.html (дата обращения: 5 января 2026 г.).

[4] Broadcom Inc. Spring Framework. — 2025. — URL: https://spring.io/ (дата обращения: 5 января 2026 г.).

[5] BugSwarm: Mining and Continuously Growing a Dataset of Reproducible Failures and Fixes / David A. Tomassi, Naji Dmeiri, Yichen Wang et al. // Proceedings of the 41st International Conference on Software Engineering. — ICSE '19. — Piscataway, NJ, USA : IEEE Press, 2019. — P. 339–349. — URL: https://doi.org/10.1109/ICSE.2019.00048 (дата обращения: 5 января 2026 г.).

[6] Chiba Shigeru. Javassist: Java Bytecode Engineering Toolkit. — GitHub repository. — 2023. — URL: https://www.javassist.org/ (дата обращения: 5 января 2026 г.).

[7] Cloud Native Computing Foundation. OpenTelemetry for Java. — 2025. — URL: https://opentelemetry.io/docs/instrumentation/java/ (дата обращения: 5 января 2026 г.).

[8] Quan Shanghaoran, Yang Jiaxi, Yu Bowen et al. CodeElo: Benchmarking Competition-level Code Generation of LLMs with Human-comparable Elo Ratings. — 2025. — 2501.01257.

[9] Eclipse Foundation. AspectJ. — 2025. — URL: https://eclipse.dev/aspectj/ (дата обращения: 5 января 2026 г.).

[10] Chen Mark, Tworek Jerry, Jun Heewoo et al. Evaluating Large Language Models Trained on Code. — 2021. — 2107.03374.

[11] Explainable Automated Debugging via Large Language Model-driven Scientific Debugging / Sungmin Kang, Bei Chen, Shin Yoo, Jian-Guang Lou // Empirical Software Engineering. — 2025. — Vol. 30, no. 45. — P. 1–28. — URL: https://link.springer.com/article/10.1007/s10664-024-10594-x (дата обращения: 5 января 2026 г.).

[12] Arab Maryam, Liang Jenny T., Hong Valentina, LaToza Thomas D. How Developers Choose Debugging Strategies for Challenging Web Application Defects. — 2025. — 2501.11792.

[13] Wang Yunkun, Zhang Yue, Li Guochang et al. InspectCoder: Dynamic Analysis-Enabled Self Repair through Interactive LLM-Debugger Collaboration. — 2025. — 2510.18327.

[14] Volkov Alexey, Volkov Ivan, Stepanov Daniil et al. JacoDB: Java Compilation Database. — GitHub repository. — 2024. — URL: https://github.com/UnitTestBot/jacodb (дата обращения: 5 января 2026 г.).

[15] JetBrains. IntelliJ IDEA. — 2025. — URL: https://jetbrains.com/idea/ (дата обращения: 5 января 2026 г.).

[16] Kulikov Egor, Muravev Ilya. JPoint 2026. Миссия невыполнима? Глубокая отладка Java- и Kotlin- приложений с помощью AI. — 2026. — URL: https://jpoint.ru/talks/9b7b85ab210646cb91f1002c44388ba7/ (дата обращения: 2 мая 2026 г.).

[17] LLMDB. CodeForces Benchmark. — 2025. — URL: https://llmdb.com/benchmarks/codeforces (дата обращения: 5 января 2026 г.).

[18] Ni Ansong, Allamanis Miltiadis, Cohan Arman et al. NExT: Teaching Large Language Models to Reason about Code Execution. — 2024. — 2404.14662.

[19] OW2 Consortium. ASM: A Java Bytecode Manipulation Framework. — OW2 project. — 2025. — URL: https://asm.ow2.io/ (дата обращения: 5 января 2026 г.).

[20] OpenAI. A Practical Guide to Building Agents. — 2025. — URL: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf (дата обращения: 5 января 2026 г.).

[21] Oracle Corporation. Java Debug Interface (JDI). — 2014. — URL: https://docs.oracle.com/javase/8/docs/jdk/api/jpda/jdi/ (дата обращения: 5 января 2026 г.).

[22] Paulsen Norman. Context Is What You Need: The Maximum Effective Context Window for Real World Limits of LLMs. — 2025. — 2509.21361.

[23] Prenner Julian Aron, Robbes Romain. RunBugRun – An Executable Dataset for Automated Program Repair. — 2023. — 2304.01102.

[24] Austin Jacob, Odena Augustus, Nye Maxwell et al. Program Synthesis with Large Language Models. — 2021. — 2108.07732.

[25] ROSE: An IDE-Based Interactive Repair Framework for Debugging / Steven P. Reiss, Xuan Wei, Jiahao Yuan, Qi Xin // ACM Transactions on Software Engineering and Methodology. — 2024. — Vol. 34, no. 4. — 39 p. — URL: https://doi.org/10.1145/3705306 (дата обращения: 5 января 2026 г.).

[26] Re-factoring based Program Repair applied to Programming Assignments / Yang Hu, Umair Z. Ahmed, Sergey Mechtaev et al. // 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE). — IEEE/ACM, 2019. — P. 388–398. — URL: https://doi.org/10.1109/ASE.2019.00044 (дата обращения: 5 января 2026 г.).

[27] SWE-bench: Can Language Models Resolve Real-World GitHub Issues? / Carlos E. Jimenez, John Yang, Alexander Wettig et al. // Proceedings of the 12th International Conference on Learning Representations (ICLR 2024). — 2024. — 2310.06770.

[28] Sennrich Rico, Haddow Barry, Birch Alexandra. Neural Machine Translation of Rare Words with Subword Units // Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). — Berlin, Germany : Association for Computational Linguistics, 2016. — P. 1715–1725. — URL: https://aclanthology.org/P16-1162 (дата обращения: 9 мая 2026 г.).

[29] Arzt Steven, Lhoták Ondřej, Benz Manuel et al. Soot: A Java Optimization Framework. — GitHub repository. — 2025. — URL: https://github.com/soot-oss/soot (дата обращения: 5 января 2026 г.).

[30] A Survey on Large Language Models for Code Generation / Juyong Jiang, Fan Wang, Jiasi Shen et al. // ACM Transactions on Software Engineering and Methodology. — 2024. — URL: https://doi.org/10.1145/3747588 (дата обращения: 5 января 2026 г.).

[31] Haque Mirazul, Babkin Petr, Farmahinifarahani Farima, Veloso Manuela. Towards Effectively Leveraging Execution Traces for Program Repair with Code LLMs. — 2025. — May. — 2505.04441.

[32] UnitTestBot. UTBotJava: Automated Unit Test Generation and Precise Code Analysis for Java. — GitHub repository. — 2025. — URL: https://github.com/UnitTestBot/UTBotJava (дата обращения: 5 января 2026 г.).

[33] Unsupervised Translation of Programming Languages / Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, Guillaume Lample // Advances in Neural Information Processing Systems 33 (NeurIPS 2020). — 2020. — 2006.03511.

[34] Winterhalter Rafael. Byte Buddy: Runtime Code Generation for the Java Virtual Machine. — GitHub repository. — 2025. — URL: https://bytebuddy.net/ (дата обращения: 5 января 2026 г.).

[35] Zhong Li, Wang Zilong, Shang Jingbo. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step by Step // Findings of the Association for Computational Linguistics: ACL 2024 / Ed. by Lun-Wei Ku, Andre Martins, Vivek Srikumar. — Bangkok, Thailand : Association for Computational Linguistics, 2024. — Aug. — P. 851–870. — URL: https://aclanthology.org/2024.findings-acl.49/ (дата обращения: 5 января 2026 г.).

[36] ООО «ИИТЕХ». Veai. — 2025. — URL: https://veai.ru/ (дата обращения: 5 января 2026 г.).

[37] ООО «ИИТЕХ». Матрица функциональности Veai. — 2025. — URL: https://veai.ru/docs/veai/feature-matrix (дата обращения: 5 января 2026 г.).